

DATA STRUCTURES (LAB)

LAB ASSIGNMENT 01



By

MISBAH KHALID

FA25-BCS-005

Submitted to,

Mr. BILAL BUKHARI

OCTOBER 4TH, 2026

Department of Computer Sciences
COMSATS University Islamabad,
Attock Campus

SINGLY LINK LIST

TASK 01

CODE:

```
#include <iostream>
using namespace std;

// Node structure as per lab manual
struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

// Insert at end
void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;

    if (first == NULL) {
        first = last = p;
    } else {
        last->next = p;
        last = p;
    }
}

// Recursive display in reverse
void displayReverseRecursive(NodeType *p) {
    if (p == NULL) return;
    displayReverseRecursive(p->next); // go to end first
    cout << p->data << " ";        // print on way back
}

// Iterative reverse display using recursion internally
void displayReverse() {
    cout << "List in reverse: ";
    displayReverseRecursive(first);
    cout << endl;
}

int main() {
    insert_end(100); // oldest
    insert_end(200);
    insert_end(300);
    insert_end(400); // newest
```

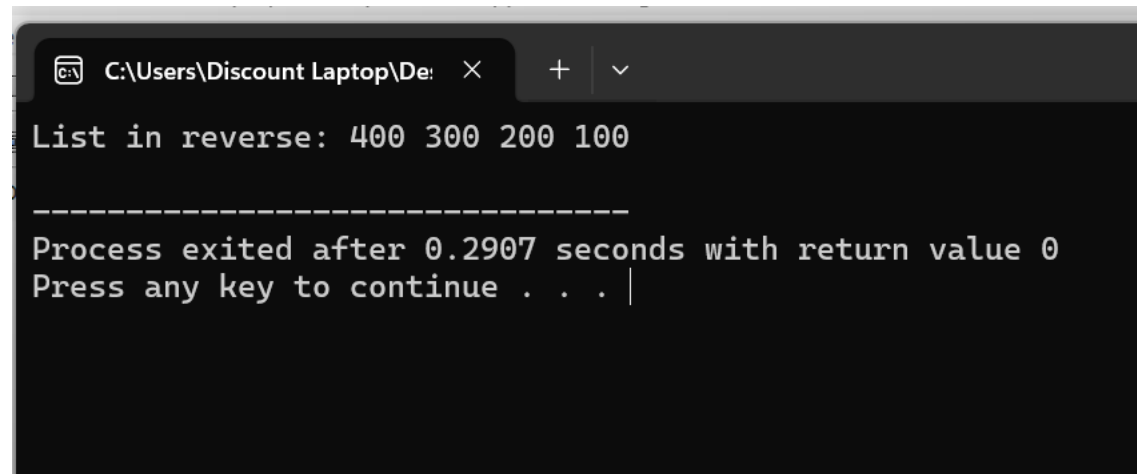
```

    displayReverse(); // 400 300 200 100

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De:
List in reverse: 400 300 200 100
-----
Process exited after 0.2907 seconds with return value 0
Press any key to continue . . . |

```

TASK 02

CODE

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Reverse list by changing links only (addresses remain same)
void reverseList() {
    NodeType *prev = NULL;
    NodeType *current = first;
    NodeType *nextNode = NULL;

    last = first; // old first becomes new last
}

```

```

while (current != NULL) {
    nextNode = current->next;
    current->next = prev;
    prev = current;
    current = nextNode;
}
first = prev;
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}

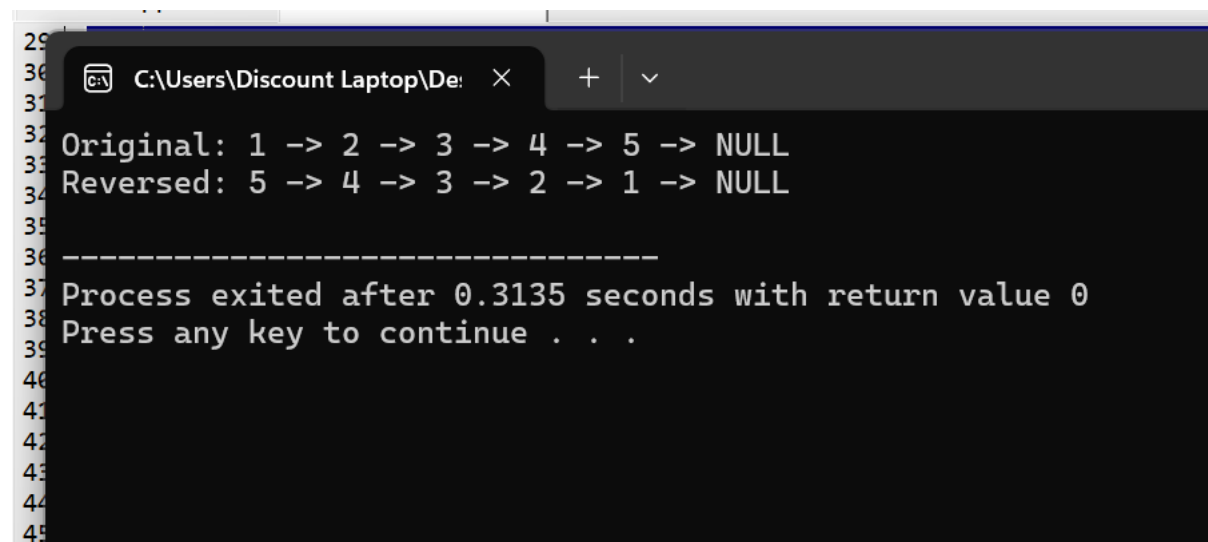
int main() {
    insert_end(1); insert_end(2); insert_end(3);
    insert_end(4); insert_end(5);

    cout << "Original: "; display();
    reverseList();
    cout << "Reversed: "; display();

    return 0;
}

```

OUTPUT:



```

29
30
31
32 Original: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
33 Reversed: 5 -> 4 -> 3 -> 2 -> 1 -> NULL
34
35 -----
36
37 Process exited after 0.3135 seconds with return value 0
38 Press any key to continue . . .
39
40
41
42
43
44
45

```

TASK 03

CODE

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Remove duplicates without extra data structure
void removeDuplicates() {
    NodeType *current = first;

    while (current != NULL && current->next != NULL) {
        NodeType *runner = current;

        while (runner->next != NULL) {
            if (runner->next->data == current->data) {
                NodeType *dup = runner->next;
                runner->next = runner->next->next;
                if (dup == last) last = runner;
                delete dup;
            } else {
                runner = runner->next;
            }
        }
        current = current->next;
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}
```

```

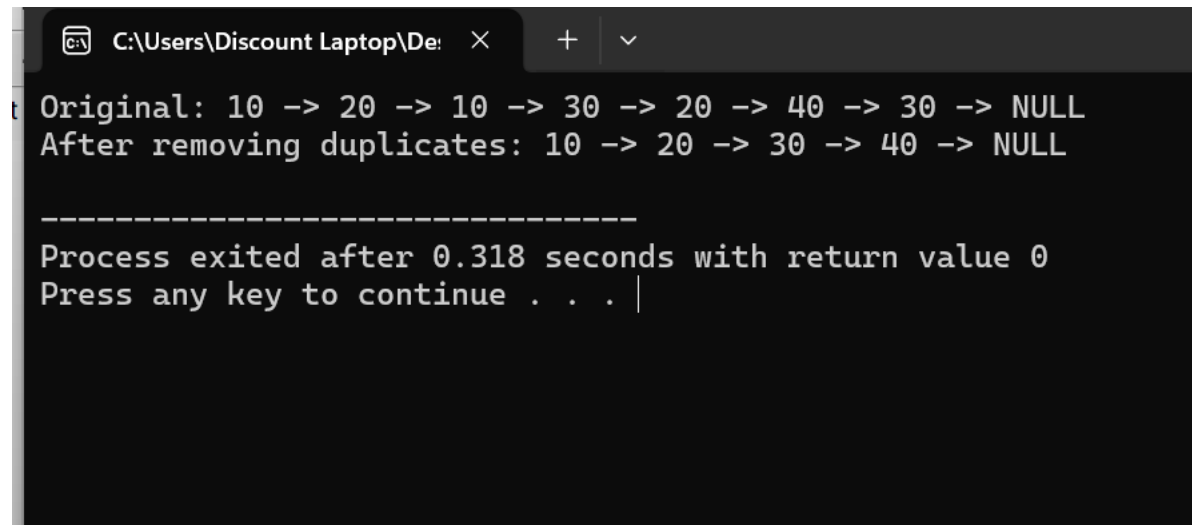
int main() {
    insert_end(10); insert_end(20); insert_end(10);
    insert_end(30); insert_end(20); insert_end(40);
    insert_end(30);

    cout << "Original: "; display();
    removeDuplicates();
    cout << "After removing duplicates: "; display();

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De: × + ▾
Original: 10 -> 20 -> 10 -> 30 -> 20 -> 40 -> 30 -> NULL
After removing duplicates: 10 -> 20 -> 30 -> 40 -> NULL

-----
Process exited after 0.318 seconds with return value 0
Press any key to continue . . . |

```

TASK 04

CODE

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

```

```

// Create loop for testing
void createLoop(int pos) {
    NodeType *loopNode = first;
    for (int i = 0; i < pos && loopNode != NULL; i++)
        loopNode = loopNode->next;
    if (loopNode != NULL) last->next = loopNode;
}

// Floyd's Cycle Detection (Tortoise & Hare)
bool detectLoop() {
    NodeType *slow = first;
    NodeType *fast = first;

    while (slow != NULL && fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

int main() {
    insert_end(1); insert_end(2); insert_end(3);
    insert_end(4); insert_end(5);

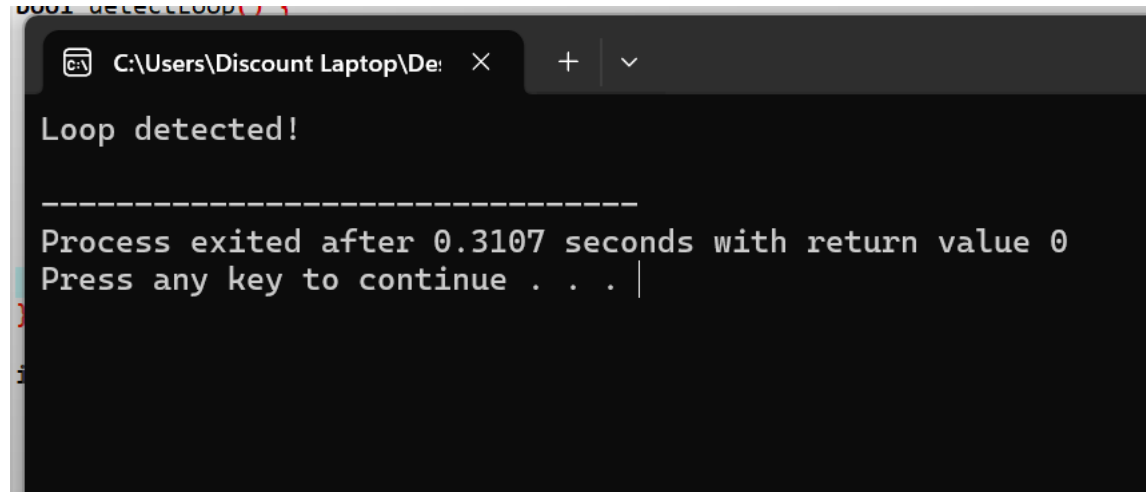
    createLoop(2); // loop back to node index 2

    if (detectLoop())
        cout << "Loop detected!" << endl;
    else
        cout << "No loop found." << endl;

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De: x + v
Loop detected!
-----
Process exited after 0.3107 seconds with return value 0
Press any key to continue . . . |

```

TASK 05

CODE

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

// Function to find middle node
Node* findMiddle(Node* head) {
    if (head == NULL)
        return NULL;

    Node* slow = head;
    Node* fast = head;

    // Find left middle for even number of nodes
    while (fast->next != NULL && fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

// Insert node at end
void insertEnd(Node*& head, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}

int main() {
    Node* head = NULL;
```

```

insertEnd(head, 1);
insertEnd(head, 2);
insertEnd(head, 3);
insertEnd(head, 4);

```

```

Node* middle = findMiddle(head);

```

```

if (middle != NULL)
    cout << "Middle node: " << middle->data;

```

```

return 0;

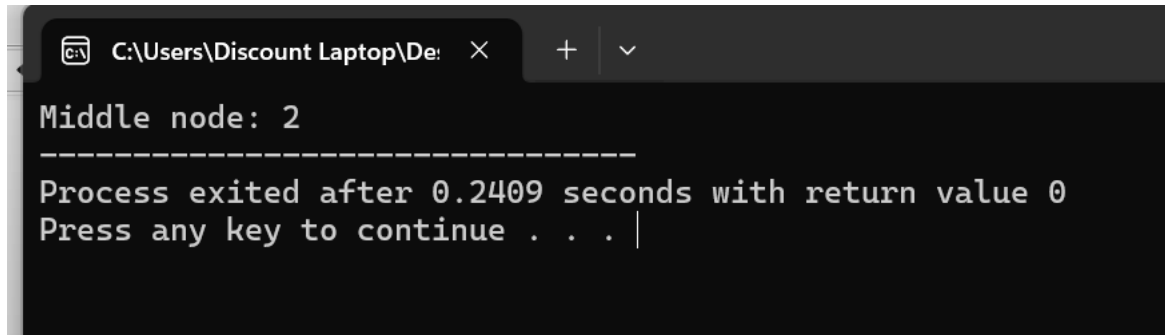
```

```

}

```

OUTPUT



```

C:\Users\Discount Laptop\De...
Middle node: 2
-----
Process exited after 0.2409 seconds with return value 0
Press any key to continue . . . |

```

TASK 06

CODE

```

#include <iostream>
using namespace std;

```

```

struct NodeType {
    int data;
    NodeType *next;
};

```

```

NodeType *first = NULL, *last = NULL;

```

```

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

```

```

// Separate into even and odd lists

```

```

void separateEvenOdd(NodeType *&evenHead, NodeType *&oddHead) {
    evenHead = oddHead = NULL;
}

```

```

NodeType *evenTail = NULL, *oddTail = NULL;

NodeType *p = first;
while (p != NULL) {
    NodeType *nextNode = p->next;
    p->next = NULL; // detach

    if (p->data % 2 == 0) {
        if (evenHead == NULL) evenHead = evenTail = p;
        else { evenTail->next = p; evenTail = p; }
    } else {
        if (oddHead == NULL) oddHead = oddTail = p;
        else { oddTail->next = p; oddTail = p; }
    }
    p = nextNode;
}

void displayList(NodeType *head, string name) {
    cout << name << ": ";
    NodeType *p = head;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}

int main() {
    insert_end(11); insert_end(20); insert_end(33);
    insert_end(40); insert_end(55); insert_end(60);

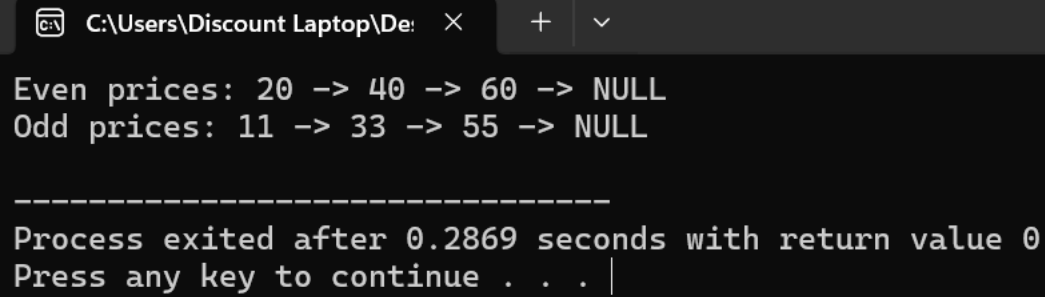
    NodeType *evenHead, *oddHead;
    separateEvenOdd(evenHead, oddHead);

    displayList(evenHead, "Even prices");
    displayList(oddHead, "Odd prices");

    return 0;
}

```

OUTPUT:



```
C:\Users\Discount Laptop\De: X + v

Even prices: 20 -> 40 -> 60 -> NULL
Odd prices: 11 -> 33 -> 55 -> NULL

-----

Process exited after 0.2869 seconds with return value 0
Press any key to continue . . . |
```

TASK 07

CODE

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Reverse a segment from start to end
NodeType* reverseSegment(NodeType *start, NodeType *end) {
    NodeType *prev = NULL;
    NodeType *curr = start;
    NodeType *stop = end->next;

    while (curr != stop) {
        NodeType *nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;
}
```

```

// Reverse halves separately
void reverseHalves() {
    if (first == NULL || first->next == NULL) return;

    // Find middle
    NodeType *slow = first, *fast = first;
    NodeType *prevSlow = NULL;

    while (fast != NULL && fast->next != NULL) {
        prevSlow = slow;
        slow = slow->next;
        fast = fast->next->next;
    }

    NodeType *firstHalfStart = first;
    NodeType *firstHalfEnd = prevSlow;
    NodeType *secondHalfStart = slow;

    NodeType *secondHalfEnd = secondHalfStart;
    while (secondHalfEnd->next != NULL)
        secondHalfEnd = secondHalfEnd->next;

    // Reverse each half
    NodeType *newFirstHead = reverseSegment(firstHalfStart, firstHalfEnd);
    NodeType *newSecondHead = reverseSegment(secondHalfStart, secondHalfEnd);

    // Connect
    firstHalfStart->next = newSecondHead;
    first = newFirstHead;
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data;
        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    for (int i = 1; i <= 8; i++) insert_end(i);

    cout << "Original: "; display();
    reverseHalves();
    cout << "After: "; display();

    return 0;
}

```

```
}
```

OUTPUT;

```
C:\Users\Discount Laptop\De: × + v
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
After: 4 -> 3 -> 2 -> 1 -> 8 -> 7 -> 6 -> 5

-----
Process exited after 0.2861 seconds with return value 0
Press any key to continue . . . |
```

TASK 08

CODE

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

void removeDuplicates() {
    NodeType *current = first;
    while (current != NULL && current->next != NULL) {
        NodeType *runner = current;
        while (runner->next != NULL) {
            if (runner->next->data == current->data) {
                NodeType *dup = runner->next;
                runner->next = runner->next->next;
                if (dup == last) last = runner;
            }
        }
        current = current->next;
    }
}
```

```

        delete dup;
    } else {
        runner = runner->next;
    }
}
current = current->next;
}
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}

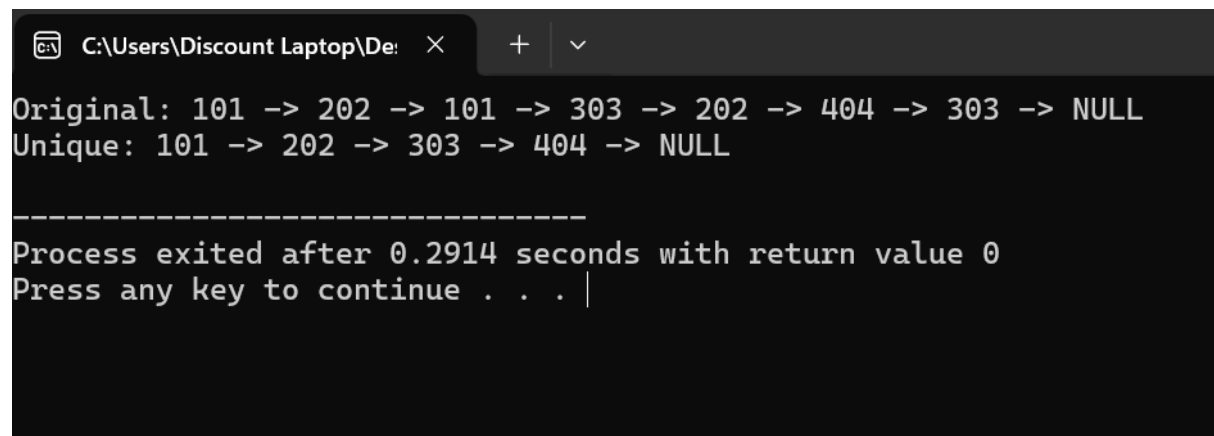
int main() {
    insert_end(101); insert_end(202); insert_end(101);
    insert_end(303); insert_end(202); insert_end(404);
    insert_end(303);

    cout << "Original: "; display();
    removeDuplicates();
    cout << "Unique: "; display();

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De:  X  +  v
Original: 101 -> 202 -> 101 -> 303 -> 202 -> 404 -> 303 -> NULL
Unique: 101 -> 202 -> 303 -> 404 -> NULL

-----
Process exited after 0.2914 seconds with return value 0
Press any key to continue . . . |

```

TASK 09

CODE

```

#include <iostream>
using namespace std;

```

```

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Delete all occurrences of a given ID
void deleteAllInstances(int bookID) {
    // Handle head deletions
    while (first != NULL && first->data == bookID) {
        NodeType *temp = first;
        first = first->next;
        delete temp;
    }
    if (first == NULL) { last = NULL; return; }

    // Handle middle and end
    NodeType *current = first;
    while (current != NULL && current->next != NULL) {
        if (current->next->data == bookID) {
            NodeType *temp = current->next;
            current->next = current->next->next;
            if (temp == last) last = current;
            delete temp;
        } else {
            current = current->next;
        }
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data << " -> ";
        p = p->next;
    }
    cout << "NULL" << endl;
}

int main() {
    insert_end(5); insert_end(10); insert_end(5);
}

```

```

insert_end(20); insert_end(5); insert_end(30);
insert_end(5);

cout << "Original: "; display();
deleteAllInstances(5);
cout << "After deleting 5: "; display();

return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De:
Original: 5 -> 10 -> 5 -> 20 -> 5 -> 30 -> 5 -> NULL
After deleting 5: 10 -> 20 -> 30 -> NULL

-----
Process exited after 0.3065 seconds with return value 0
Press any key to continue . . . |

```

TASK 10

CODE

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int value) {
    NodeType *p = new NodeType;
    p->data = value;
    p->next = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; last = p; }
}

// Swap every two consecutive nodes
void swapPairs() {
    if (first == NULL || first->next == NULL) return;

    // Dummy node simplifies edge cases
    NodeType *dummy = new NodeType;
    dummy->next = first;
    NodeType *prev = dummy;

```

```

while (prev->next != NULL && prev->next->next != NULL) {
    NodeType *firstNode = prev->next;
    NodeType *secondNode = firstNode->next;

    // Swap
    firstNode->next = secondNode->next;
    secondNode->next = firstNode;
    prev->next = secondNode;

    prev = firstNode;
}

first = dummy->next;
delete dummy;

// Update last pointer
last = first;
while (last != NULL && last->next != NULL)
    last = last->next;
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data;
        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << " -> NULL" << endl;
}

int main() {
    insert_end(1); insert_end(2); insert_end(3);
    insert_end(4); insert_end(5); insert_end(6);

    cout << "Original: "; display();
    swapPairs();
    cout << "Swapped: "; display();

    return 0;
}

```

OUTPUT:

```
C:\Users\Discount Laptop\De: X + v
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> NULL
Swapped: 2 -> 1 -> 4 -> 3 -> 6 -> 5 -> NULL

-----
Process exited after 0.299 seconds with return value 0
Press any key to continue . . . |
```

DOUBLY LINK LIST

TASK 01

CODE

```
#include <iostream>
#include <string>
using namespace std;

struct NodeType {
    string name;
    NodeType *next;
    NodeType *prev;
};

NodeType *first = NULL, *last = NULL;

void insert_end(string val) {
    NodeType *p = new NodeType;
    p->name = val;
    p->next = NULL;
    p->prev = NULL;

    if (first == NULL) {
        first = last = p;
    } else {
        last->next = p;
        p->prev = last;
        last = p;
    }
}
```

```

// Swap values from both ends toward center
void swapEndsTowardsCenter() {
    if (first == NULL || first == last) return;

    NodeType *left = first;
    NodeType *right = last;

    while (left != right && left->prev != right) {
        swap(left->name, right->name);
        left = left->next;
        right = right->prev;
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->name;
        if (p->next != NULL) cout << " <-> ";
        p = p->next;
    }
    cout << endl;
}

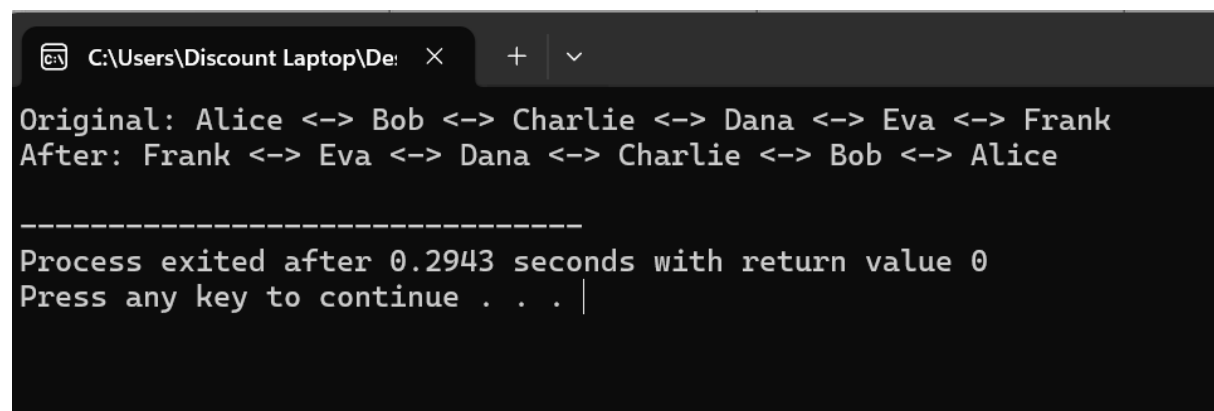
int main() {
    insert_end("Alice"); insert_end("Bob"); insert_end("Charlie");
    insert_end("Dana"); insert_end("Eva"); insert_end("Frank");

    cout << "Original: "; display();
    swapEndsTowardsCenter();
    cout << "After: "; display();

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De...
Original: Alice <-> Bob <-> Charlie <-> Dana <-> Eva <-> Frank
After: Frank <-> Eva <-> Dana <-> Charlie <-> Bob <-> Alice

-----
Process exited after 0.2943 seconds with return value 0
Press any key to continue . . . |

```

TASK 02

CODE

```
#include <iostream>
#include <string>
using namespace std;

struct Song {
    string title;
    string artist;
    Song *next;
    Song *prev;
};

Song *head = NULL, *tail = NULL, *current = NULL;

void addSong(string t, string a) {
    Song *p = new Song;
    p->title = t;
    p->artist = a;
    p->next = NULL;
    p->prev = NULL;

    if (head == NULL) {
        head = tail = current = p;
    } else {
        tail->next = p;
        p->prev = tail;
        tail = p;
    }
}

void nextSong() {
    if (current == NULL) { cout << "Playlist empty\n"; return; }
    if (current->next == NULL) current = head;
    else current = current->next;
    cout << "Now playing: " << current->title << " by " << current->artist << endl;
}

void prevSong() {
    if (current == NULL) { cout << "Playlist empty\n"; return; }
    if (current->prev == NULL) current = tail;
    else current = current->prev;
    cout << "Now playing: " << current->title << " by " << current->artist << endl;
}

void displayForward() {
    cout << "Playlist (forward): ";
    Song *p = head;
    while (p != NULL) {
        cout << p->title;
```

```

        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}

void displayBackward() {
    cout << "Playlist (backward): ";
    Song *p = tail;
    while (p != NULL) {
        cout << p->title;
        if (p->prev != NULL) cout << " <- ";
        p = p->prev;
    }
    cout << endl;
}

void removeSong(string title) {
    Song *p = head;
    while (p != NULL) {
        if (p->title == title) {
            if (current == p)
                current = (p->next != NULL) ? p->next : p->prev;

            if (p->prev != NULL) p->prev->next = p->next;
            else head = p->next;

            if (p->next != NULL) p->next->prev = p->prev;
            else tail = p->prev;

            delete p;
            cout << "Removed: " << title << endl;
            return;
        }
        p = p->next;
    }
    cout << "Song not found: " << title << endl;
}

int main() {
    addSong("Song A", "Artist 1");
    addSong("Song B", "Artist 2");
    addSong("Song C", "Artist 3");
    addSong("Song D", "Artist 4");

    displayForward();
    displayBackward();

    cout << "\n--- Playback ---" << endl;
    nextSong();
}

```

```

    nextSong();
    prevSong();

    cout << "\n--- Remove ---" << endl;
    removeSong("Song B");
    displayForward();

    return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De:
Playlist (forward): Song A -> Song B -> Song C -> Song D
Playlist (backward): Song D <- Song C <- Song B <- Song A

--- Playback ---
Now playing: Song B by Artist 2
Now playing: Song C by Artist 3
Now playing: Song B by Artist 2

--- Remove ---
Removed: Song B
Playlist (forward): Song A -> Song C -> Song D

-----
Process exited after 0.6399 seconds with return value 0
Press any key to continue . . . |

```

TASK 03

CODE

```

#include <iostream>
#include <string>
using namespace std;

// Item node (bottom level)
struct Item {
    string name;
    int quantity;
    Item *next;
};

// Section node (middle level)
struct Section {
    string sectionName;
    Section *next;
    Item *itemsHead;
};

// Store node (top level)

```

```

struct Store {
    string storeName;
    Store *next;
    Section *sectionsHead;
};

Store *storesHead = NULL;

Store* findStore(string name) {
    Store *p = storesHead;
    while (p != NULL) {
        if (p->storeName == name) return p;
        p = p->next;
    }
    return NULL;
}

Section* findSection(Store *store, string name) {
    if (store == NULL) return NULL;
    Section *p = store->sectionsHead;
    while (p != NULL) {
        if (p->sectionName == name) return p;
        p = p->next;
    }
    return NULL;
}

// Add a new store
void addStore(string name) {
    Store *p = new Store;
    p->storeName = name;
    p->next = NULL;
    p->sectionsHead = NULL;

    if (storesHead == NULL) storesHead = p;
    else {
        Store *temp = storesHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
    cout << "Store added: " << name << endl;
}

// Add a new section in a store
void addSection(string storeName, string sectionName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    if (findSection(store, sectionName) != NULL) {
        cout << "Section already exists\n";
    }
}

```

```

        return;
    }

    Section *p = new Section;
    p->sectionName = sectionName;
    p->next = NULL;
    p->itemsHead = NULL;

    if (store->sectionsHead == NULL) store->sectionsHead = p;
    else {
        Section *temp = store->sectionsHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
    cout << "Section added: " << sectionName << " in " << storeName << endl;
}

// Store an item in a particular section of a particular store
void addItem(string storeName, string sectionName, string itemName, int qty) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    Item *p = new Item;
    p->name = itemName;
    p->quantity = qty;
    p->next = NULL;

    if (section->itemsHead == NULL) section->itemsHead = p;
    else {
        Item *temp = section->itemsHead;
        while (temp->next != NULL) temp = temp->next;
        temp->next = p;
    }
    cout << "Item added: " << itemName << " (" << qty << ") in "
        << storeName << " -> " << sectionName << endl;
}

// Remove an item from a particular section of a particular store
void removeItem(string storeName, string sectionName, string itemName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    Item *current = section->itemsHead;
    Item *prev = NULL;

```

```

while (current != NULL) {
    if (current->name == itemName) {
        if (prev == NULL) section->itemsHead = current->next;
        else prev->next = current->next;
        delete current;
        cout << "Item removed: " << itemName << endl;
        return;
    }
    prev = current;
    current = current->next;
}
cout << "Item not found: " << itemName << endl;
}

// Display list of all items of a particular section of a store
void displaySectionItems(string storeName, string sectionName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    Section *section = findSection(store, sectionName);
    if (section == NULL) { cout << "Section not found\n"; return; }

    cout << "Items in " << storeName << " -> " << sectionName << ": ";
    Item *p = section->itemsHead;
    while (p != NULL) {
        cout << p->name << "(" << p->quantity << ") ";
        p = p->next;
    }
    cout << endl;
}

// Display list of items for a given store
void displayStoreItems(string storeName) {
    Store *store = findStore(storeName);
    if (store == NULL) { cout << "Store not found\n"; return; }

    cout << "Store: " << storeName << endl;
    Section *s = store->sectionsHead;
    while (s != NULL) {
        cout << " Section " << s->sectionName << ": ";
        Item *p = s->itemsHead;
        while (p != NULL) {
            cout << p->name << "(" << p->quantity << ") ";
            p = p->next;
        }
        cout << endl;
        s = s->next;
    }
}

```

```

int main() {
    addStore("Walmart");
    addStore("Target");

    addSection("Walmart", "Toys");
    addSection("Walmart", "Grocery");
    addSection("Target", "Fruits");

    addItem("Walmart", "Toys", "Lego", 50);
    addItem("Walmart", "Toys", "Doll", 30);
    addItem("Walmart", "Grocery", "Milk", 100);
    addItem("Target", "Fruits", "Apple", 200);

    displaySectionItems("Walmart", "Toys");
    displayStoreItems("Walmart");

    removeItem("Walmart", "Toys", "Doll");
    displaySectionItems("Walmart", "Toys");

    return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De:
Store added: Walmart
Store added: Target
Section added: Toys in Walmart
Section added: Grocery in Walmart
Section added: Fruits in Target
Item added: Lego (50) in Walmart -> Toys
Item added: Doll (30) in Walmart -> Toys
Item added: Milk (100) in Walmart -> Grocery
Item added: Apple (200) in Target -> Fruits
Items in Walmart -> Toys: Lego(50) Doll(30)
Store: Walmart
    Section Toys: Lego(50) Doll(30)
    Section Grocery: Milk(100)
Item removed: Doll
Items in Walmart -> Toys: Lego(50)

-----
Process exited after 0.4114 seconds with return value 0
Press any key to continue . . .

```

TASK 04

CODE:

```
#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
    NodeType *prev;
};

NodeType *first = NULL, *last = NULL;

void insert_end(int val) {
    NodeType *p = new NodeType;
    p->data = val;
    p->next = NULL;
    p->prev = NULL;
    if (first == NULL) first = last = p;
    else { last->next = p; p->prev = last; last = p; }
}

// Pattern: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9
// First and last remain fixed; alternate swaps toward center
void specialPattern() {
    if (first == NULL || first == last) return;

    NodeType *left = first->next;    // start after first
    NodeType *right = last->prev;    // start before last

    bool swapFlag = true;

    while (left != right && left->prev != right) {
        if (swapFlag) {
            swap(left->data, right->data);
            left = left->next;
            right = right->prev;
        } else {
            left = left->next;
            right = right->prev;
        }
        swapFlag = !swapFlag;
    }
}

void display() {
    NodeType *p = first;
    while (p != NULL) {
        cout << p->data;
```

```

        if (p->next != NULL) cout << " -> ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    for (int i = 1; i <= 9; i++) insert_end(i);

    cout << "Original: "; display();
    specialPattern();
    cout << "After: "; display();

    return 0;
}

```

OUTPUT:

```

C:\Users\Discount Laptop\De:  X  +  v
Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
After: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9

-----
Process exited after 0.2984 seconds with return value 0
Press any key to continue . . . |

```

CIRCULAR LINK LIST

TASK 01

CODE

```

#include <iostream>
using namespace std;

struct NodeType {
    int data;
    NodeType *next;
};

NodeType *pNode = NULL;

// Insert into circular linked list (as per Lab 04 Activity 1)
void insert(int value) {
    NodeType *p = new NodeType;
    p->data = value;

    if (pNode == NULL) {
        p->next = p;
        pNode = p;
    } else {
        p->next = pNode->next;
    }
}

```

```

        pNode->next = p;
        pNode = p;
    }
}

```

// Josephus Problem Solution

```

int josephus(int N, int M) {
    pNode = NULL;

```

```

    // Create circle with N persons numbered 1 to N
    for (int i = 1; i <= N; i++) {
        insert(i);
    }

```

// Start from first person

```

NodeType *current = pNode->next; // points to person 1
NodeType *prev = pNode;         // points to person N

```

```

cout << "Elimination order: ";

```

// Eliminate until only one remains

```

while (current->next != current) {
    // Skip M-1 persons
    for (int count = 1; count < M; count++) {
        prev = current;
        current = current->next;
    }

```

// Kill Mth person

```

cout << current->data << " ";
prev->next = current->next;

```

// If current was pNode, update pNode

```

if (current == pNode) {
    pNode = prev;
}

```

```

delete current;
current = prev->next;

```

```

}

cout << endl;
int survivor = current->data;
delete current;
pNode = NULL;
return survivor;
}

```

```

int main() {
    int N, M;

```

```

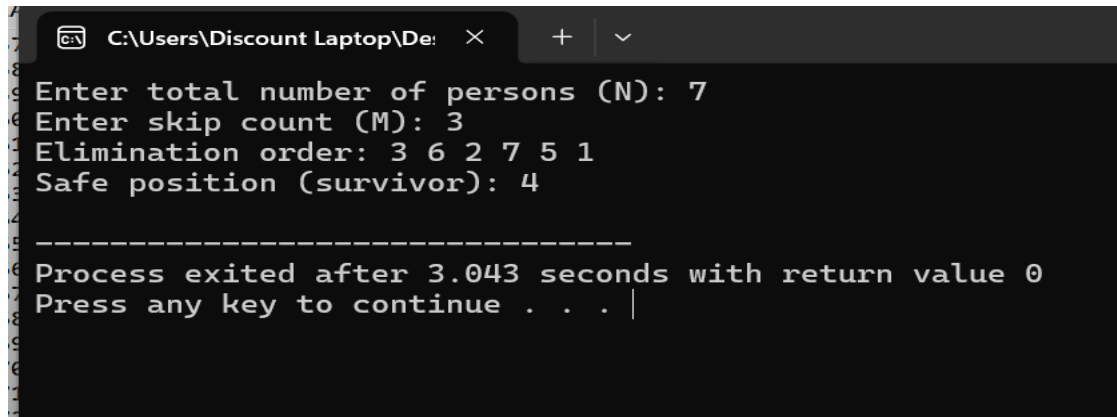
    cout << "Enter total number of persons (N): ";
    cin >> N;
    cout << "Enter skip count (M): ";
    cin >> M;

    int safePosition = josephus(N, M);
    cout << "Safe position (survivor): " << safePosition << endl;

    return 0;
}

```

OUTPUT:



```

C:\Users\Discount Laptop\De:
Enter total number of persons (N): 7
Enter skip count (M): 3
Elimination order: 3 6 2 7 5 1
Safe position (survivor): 4
-----
Process exited after 3.043 seconds with return value 0
Press any key to continue . . . |

```

TASK 02

TASK SCHEDULING SYSTEM

CODE

```

#include <iostream>
#include <string>
using namespace std;

struct Task {
    string name;
    int priority;
    string status; // "pending", "in-progress", "completed"
    Task *next;
};

Task *head = NULL;
Task *currentTask = NULL; // for round-robin

// Add a task at end (circular)
void addTask(string name, int priority) {
    Task *p = new Task;
    p->name = name;
    p->priority = priority;
    p->status = "pending";
}

```

```

if (head == NULL) {
    p->next = p;
    head = p;
} else {
    Task *temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    temp->next = p;
    p->next = head;
}
cout << "Task added: " << name << endl;
}

// Remove a task by name
void removeTask(string name) {
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }

    Task *current = head;
    Task *prev = NULL;

    // Find last node (for head deletion handling)
    Task *last = head;
    while (last->next != head) {
        last = last->next;
    }

    do {
        if (current->name == name) {
            if (current == head) {
                if (head->next == head) {
                    // Only one node
                    delete head;
                    head = NULL;
                    currentTask = NULL;
                    cout << "Task removed: " << name << endl;
                    return;
                } else {
                    // Head deletion with multiple nodes
                    head = head->next;
                    last->next = head;
                    if (currentTask == current) {
                        currentTask = head;
                    }
                }
                delete current;
                cout << "Task removed: " << name << endl;
            }

```

```

        return;
    }
} else {
    // Middle or last node deletion
    prev->next = current->next;
    if (currentTask == current) {
        currentTask = current->next;
    }
    delete current;
    cout << "Task removed: " << name << endl;
    return;
}
}
prev = current;
current = current->next;
} while (current != head);

cout << "Task not found: " << name << endl;
}

// Get next pending task (round-robin)
void getNextTask() {
    if (head == NULL) {
        cout << "No tasks available." << endl;
        return;
    }

    if (currentTask == NULL) {
        currentTask = head;
    } else {
        currentTask = currentTask->next;
    }

    // Skip completed tasks
    Task *start = currentTask;
    while (currentTask->status == "completed") {
        currentTask = currentTask->next;
        if (currentTask == start) {
            cout << "All tasks are completed!" << endl;
            return;
        }
    }

    cout << "Next task: " << currentTask->name
        << " (Priority: " << currentTask->priority
        << ", Status: " << currentTask->status << ")" << endl;
}

// Display all tasks
void displayAllTasks() {

```

```

if (head == NULL) {
    cout << "No tasks in the list." << endl;
    return;
}

cout << "\n--- All Tasks ---" << endl;
Task *temp = head;
do {
    cout << "Name: " << temp->name
        << " | Priority: " << temp->priority
        << " | Status: " << temp->status << endl;
    temp = temp->next;
} while (temp != head);
cout << "-----" << endl;
}

// Update task status by name
void updateTaskStatus(string name, string newStatus) {
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }

    Task *temp = head;
    do {
        if (temp->name == name) {
            temp->status = newStatus;
            cout << "Task '" << name << "' status updated to: " << newStatus << endl;
            return;
        }
        temp = temp->next;
    } while (temp != head);

    cout << "Task not found: " << name << endl;
}

int main() {
    addTask("Design", 1);
    addTask("Coding", 2);
    addTask("Testing", 3);
    addTask("Deployment", 4);

    displayAllTasks();

    cout << "\n--- Round Robin ---" << endl;
    getNextTask(); // Design
    getNextTask(); // Coding

    updateTaskStatus("Coding", "completed");
}

```

```

getNextTask(); // Testing (skips nothing yet)
getNextTask(); // Deployment
getNextTask(); // Design (wraps around)

cout << "\n--- Remove Testing ---" << endl;
removeTask("Testing");
displayAllTasks();

return 0;
}

```

OUTPUT:

```

Task added: Design
Task added: Coding
Task added: Testing
Task added: Deployment

--- All Tasks ---
Name: Design | Priority: 1 | Status: pending
Name: Coding | Priority: 2 | Status: pending
Name: Testing | Priority: 3 | Status: pending
Name: Deployment | Priority: 4 | Status: pending
-----

--- Round Robin ---
Next task: Design (Priority: 1, Status: pending)
Next task: Coding (Priority: 2, Status: pending)
Task 'Coding' status updated to: completed
Next task: Testing (Priority: 3, Status: pending)
Next task: Deployment (Priority: 4, Status: pending)
Next task: Design (Priority: 1, Status: pending)

--- Remove Testing ---
Task removed: Testing

--- All Tasks ---
Name: Design | Priority: 1 | Status: pending
Name: Coding | Priority: 2 | Status: completed
Name: Deployment | Priority: 4 | Status: pending
-----

```